7주차: 1차원·2차원 동적 계획법

같은 부분 문제의 반복 계산

인접한 점검 항목을 함께 선택할 수 없을 때 최대 가치를 구하려면 어떤 정보를 남겨야 할까요? 격자 경로와 두 이벤트 기록의 공통 순서도 작은 부분 문제로 나눌 수 있습니다.

앞에서 계산한 최적값이나 개수를 저장하고 재사용합니다. 상태의 의미에 따라 합·최소·최대 중 연산을 결정합니다.

7주차 학습 목표

13·14차시: 상태 의미·초기값·점화식·계산 순서

- 입력 계약과 경계 조건 확인
- 상태 추적과 Python 구현
- 정당성·시간·보조 공간·출력 공간 설명

도입 상황, 표와 점검 문제는 강의용 추가 예시입니다.

13차시 · 최적 부분 문제와 1차원 DP

같은 접두부·금액을 반복 계산하는 탐색을 상태 표로 바꿉니다. 무엇을 최대화하거나 최소화하는지 먼저 적어 값의 의미를 고정합니다.

선수 지식과 Python 문법

`float("inf")`는 불가능한 최소화 상태를 나타냅니다. tuple 동시 대입은 오른쪽을 이전 값으로 계산합니다. 덮어쓰기 전에 필요한 이전 두 상태를 보존합니다.

DP 표를 만들기 전의 정의

- 상태: 어떤 입력 범위의 어떤 답인가?
- 초기값: 빈 입력·금액 0·첫 행과 열
- 점화식: 마지막 선택으로 경우 분할
- 계산 순서: 필요한 이전 상태부터

동전 금액을 큰 값부터 계산하면 아직 구하지 않은 작은 금액을 읽게 됩니다. 양수 동전은 작은 금액부터의 계산을 가능하게 합니다.

198 · 문제와 상태

LeetCode 198: House Robber · Medium

인접한 위치를 함께 고르지 않고 값의 합을 최대화합니다.

`nums=[3,1,4,2] → 7`

D[i]는 앞 i개 최댓값입니다. D[0]=0, D[1]=nums[0]이며 i>=2에서 D[i]=max(D[i-1],D[i-2]+nums[i-1]) 입니다. 코드에서는 이전 두 상태를 0으로 시작해 첫 값부터 같은 갱신을 적용합니다.

198 · 구현

```
def rob(nums):
    previous_two = previous_one = 0
    for value in nums:
        previous_two, previous_one = previous_one, max(previous_one, previous_two + value)
    return previous_one
```

198 · 정당성과 비용

최적 선택은 현재 위치를 선택하거나 선택하지 않는 두 경우로 나뉩니다. 두 경우 모두 남은 접두부를 최적으로 풀어야 하므로 점화식이 모든 최적해를 포함합니다.

n=길이. O(n) 시간, O(1) 보조 공간. 선택 위치 복원은 별도 상태가 필요합니다.

322 · 문제와 상태

LeetCode 322: Coin Change · Medium

amount를 만드는 최소 동전 수를 반환하고 불가능하면 -1입니다.

`coins=[1,3,4], amount=6 → 2`

dp[0]=0, 나머지는 inf입니다. 작은 금액부터 dp[a-c]+1 중 최소를 취합니다.

322 · 구현

```
def coin_change(coins, amount):
    dp = [float("inf")] * (amount + 1)
    dp[0] = 0
    for total in range(1, amount + 1):
        for coin in coins:
            if coin <= total:
                dp[total] = min(dp[total], dp[total - coin] + 1)
    return -1 if dp[amount] == float("inf") else dp[amount]
```

322 · 정당성과 비용

최적해의 마지막 동전 c를 제거하면 a-c의 최적해가 남아야 합니다. 모든 가능한 마지막 동전을 비교하므로 최소를 놓치지 않습니다. 양수 동전이라 필요한 상태가 먼저 계산됩니다.

A=amount,m=종류 수. O(Am) 시간·O(A) 보조 공간. A의 비트 길이에 대해서는 의사다항 시간입니다.

상태 추적

a	0	1	2	3	4	5	6
dp[a],coins=1,3,4	0	1	2	1	1	2	2

각 금액은 먼저 계산한 작은 금액의 상태를 사용합니다.

실패 입력과 이해 확인

[2,1,1,2]에서 짝수 위치만 또는 홀수 위치만 합하면 최적 4를 놓칩니다.

dp[a-c]에 왜 1을 더하나요?

교사용 답은 별도 진행안에 있습니다. 코드 설명은 읽기자료의 구현과 줄별로 대조합니다.

14차시 · 격자와 접두 문자열 DP

격자 경로의 개수와 두 이벤트 문자열의 공통 순서를 구합니다. 같은 표 형태라도 합산과 최댓값의 의미를 구분해 점화식을 설계합니다.

선수 지식과 Python 문법

[[0]*(m+1) for _ in range(n+1)]는 독립 행입니다. 길이 i인 접두부의 마지막 문자는 first[i-1]입니다. 빈 접두부를 위한 0행·0열을 둡니다.

62 · 문제와 상태

LeetCode 62: Unique Paths · Medium

격자에서 오른쪽·아래로만 이동하는 경로 수를 반환합니다.

`rows=2, cols=4 → 4`

첫 행·열은 1이고 내부는 위쪽 수+왼쪽 수입니다. 행마다 새 list를 만듭니다.

62 · 구현

```
def unique_paths(rows, cols):
    dp = [[1] * cols for _ in range(rows)]
    for r in range(1, rows):
        for c in range(1, cols):
            dp[r][c] = dp[r-1][c] + dp[r][c-1]
    return dp[-1][-1]
```

62 · 정당성과 비용

각 경로의 마지막 이동은 아래 또는 오른쪽 하나입니다. 두 집합은 겹치지 않고 전체 경로를 덮으므로 수를 더합니다. 행 순서로 필요한 선행 상태가 준비됩니다.

R,C=행·열 수. O(RC) 시간·O(RC) 보조 공간. 큰 정수의 비트 비용은 별도이며 공식 답 범위에서는 작습니다.

1143 · 문제와 상태

LeetCode 1143: Longest Common Subsequence · Medium

두 문자열에서 순서를 보존하는 공통 부분수열의 최대 길이를 반환합니다.

`first="abca", second="aca" → 3`

같으면 대각선+1, 다르면 위·왼쪽 중 최대입니다. 길이 0 접두부의 행·열은 0입니다.

1143 · 구현

```
def lcs(first, second):
    rows, cols = len(first), len(second)
    dp = [[0] * (cols + 1) for _ in range(rows + 1)]
    for r in range(1, rows + 1):
        for c in range(1, cols + 1):
            if first[r-1] == second[c-1]:
                dp[r][c] = dp[r-1][c-1] + 1
            else:
                dp[r][c] = max(dp[r-1][c], dp[r][c-1])
    return dp[rows][cols]
```

1143 · 정당성과 비용

끝 문자가 같으면 두 끝을 맞춘 최적 부분수열을 택할 수 있습니다. 다르면 둘을 동시에 마지막으로 맞출 수 없어 적어도 한쪽을 생략해야 합니다. 두 경우의 최댓값을 취합니다.

n,m=문자열 길이. O(nm) 시간·O(nm) 보조 공간. 길이만 반환하며 문자열 복원은 별도입니다.

상태 추적

LCS(“ab”,”ac”)	빈 접두부	a	c
빈 접두부	0	0	0
a	0	1	1
ab	0	1	1

행과 열은 0을 포함한 접두부 길이입니다.

실패 입력과 이해 확인

`[[1]*C]*R`은 같은 행을 공유합니다. `[[1]*C for _ in range(R)]`로 만듭니다.

경로 수에서는 왜 max가 아니라 +인가요?

교사용 답은 별도 진행안에 있습니다. 코드 설명은 읽기자료의 구현과 줄별로 대조합니다.

이전 두 최적값의 보존

현재 값	갱신 전 prev_two	갱신 전 prev_one	새 최적값
3	0	0	3
1	0	3	3
4	3	3	7
2	3	7	7

오른쪽 식은 모두 갱신 전 값을 사용합니다. 현재 값을 선택하면 바로 앞 위치는 건너뜁니다.

중간 점검

1. `[2,1,1,2]`의 인접하지 않은 최대 합은?
2. 동전 `[1,3,4]`로 6을 만드는 최소 개수는?
3. 1×4와 2×4 격자의 경로 수는?
4. `abc`, `ac`의 LCS 길이는? 연속해야 하나요?

독립 실습과 다음 연결

학생 실습

힌트·해설

교사용 진행안

7주차 코드 내려받기

코드·상태 정의·정당성·복잡도·경계 입력을 함께 제출합니다. 8주차에는 주제를 숨긴 혼합 평가가 있습니다.

실행과 참고자료

주차별 코드의 code 폴더에서 실행합니다.

```
python demo_13.py
python demo_14.py
python verify.py
python verify.py --student
```

마지막 명령은 학생 파일을 구현한 뒤 실행합니다.

Erik Demaine·Srini Devadas, MIT 6.006: Introduction to Algorithms (2011).